

P0785R0: Runtime-sized arrays and a C++ wrapper

Introduction

It is occasionally useful to allocate small data structures on the stack that have a variable size defined at construction, which would otherwise only be possible by using heap allocation.

Examples:

- interfacing with the readv/writev POSIX system calls
- avoiding heap allocations, since it accesses a global resource that might be contended in a multi-threaded program
- small runtime-sized data structure to apply STL algorithms calls

The C language provides variable-length arrays (VLAs) for that purpose, with the syntax

```
void f(int n)
{
    int a[n];
}
```

Most C++ compilers implement this facility also in C++ mode.

This paper proposes to standardize a subset of the C-language VLA facility and a simple C++ class (similar to `std::initializer_list`) for the purpose of stack allocation of runtime-sized data structures.

For more detailed motivation and introductory explanations, please see the papers cited in the next section.

History

Standardizing the basic C VLA syntax (but not all the ancillary features of C VLAs) was the subject of "N3639 Runtime-sized arrays with automatic storage duration (revision 5)" (by Jens Maurer), which was voted into the C++ working paper in Bristol, April 2013.

A companion facility with a C++-style library interface called `std::dynarray` was specified in "N3662 C++ Dynamic Arrays (dynarray)" (by Lawrence Crowl and Matt Austern), which was also voted into the C++ working paper in Bristol, April 2013.

Both facilities were removed from C++14 and instead put into a separate Technical Specification in Chicago, September 2013. Authority for doing so came from National Body comment CH 2 on the C++14 Committee Draft ballot, asking for removal of any feature degrading the quality of the upcoming standard. The technical grounds for removal were the usability of `std::dynarray` for objects with any kind of storage duration, which made it look like a construction-sized `std::vector` with optional compiler optimizations, instead of a specialized stack allocation facility with a C++ look and feel that some members of the committee prefer. The working draft of the Technical Specification was published as "N3820 Working Draft, Technical Specification - Array Extensions" (editor: Lawrence Crowl).

After the Chicago meeting, there was additional exploration of the design space in the papers

- N3810 Alternatives for Array Extensions (by Bjarne Stroustrup)
- N3875 Run-time bound array data members (by Daniel Garcia and Xin Li)

The Issaquah 2014 meeting saw a lengthy discussion of the design approaches, but little actual progress.

The Array TS project was canceled in Jacksonville, March 2016.

General remarks

The C++ standard is silent on concepts such as "stack" or "heap". Instead, objects have different kinds of storage duration, depending on how they came to exist. Therefore, it would be new grounds to normatively specify that certain kinds of objects must be allocated on the stack (and not on the heap). As a quality-of-implementation concern, most implementations already do the obvious. However, implementations for special target audiences may diverge from that de-facto consensus. For example, when stack space is severely limited for environmental reasons (e.g. in kernel or embedded code), it seems plausible for a compiler to transparently allocate (and deallocate) large objects from some other pool of memory and only store a simple pointer on the stack. Therefore, the normative specification can only make sure that stack allocation is actually possible and, at best, encourage it non-normatively.

Construction of an object that is a runtime-sized data structure (in short, an "object of runtime size") requires a three-step approach: determine size, allocate storage, invoke the constructor. These are exactly the same three steps that are required for the expression `"new T[n]"`.

There are two conflicting opinions on the general design approach for a C++-style library interface (or building blocks enabling such), which cannot be reconciled:

- a best-effort class without usage restrictions that allocates from the heap unless tailored compiler optimizations turn that into allocation on the stack (`std::dynarray`)
- a class with compiler-enforced usage restrictions that make it possible to always allocate from the stack (`std::bs_array`)

(As explained in the first paragraph, it is beyond the scope of the C++ standard to actually enforce stack allocation, even in the latter case.)

Design discussion

This proposal re-proposes "N3639 Runtime-sized arrays with automatic storage duration (revision 5)", by Jens Maurer, together with a simple C++ class similar to `bs_array` proposed in "N3810 Alternatives for Array Extensions", by Bjarne Stroustrup. Please see these papers for detailed motivation and discussion.

Naming

The proposed name for the C++ class is `stack_array`.

Proposed Wording

Change in 7.2 [conv.array] paragraph 1:

An ~~lvalue or rvalue~~ **expression** of type "array of N T", **"array of runtime bound of T"**, or "array of unknown bound of T" can be converted to a prvalue of type "pointer to T". The result is a pointer to the first element of the array.

Change in 8.1.5.2 [expr.prim.lambda.capture] paragraph 11:

[...] [Note: The cast ensures that the transformed expression is a prvalue. -- end note] **An array of runtime bound (11.3.4 [dcl.array]) shall not be captured by copy.** An *id-expression* within the *compound-statement* of a *lambda-expression* that is an odr-use of a reference captured by reference refers to the entity to which the captured reference is bound and not to the captured reference. [Note: ...]

Change in 8.1.5.2 [expr.prim.lambda.capture] paragraph 12:

An entity is *captured by reference* if it is implicitly or explicitly captured but not captured by copy. It is unspecified whether additional unnamed non-static data members are declared in the closure type for entities captured by reference. If declared, such non-static data members shall be of literal type. [Example: ...] [**Note: Capturing by reference an array of runtime bound also implicitly captures the value of the bound to support the range-based for statement (9.5.4 [stmt.ranged]).**] A bit-field or a member of an anonymous union shall not be captured by reference.

Insert a new paragraph before 8.2.8 [expr.typeid] paragraph 2:

The typeid operator shall not be applied to an array of runtime bound (11.3.4 [dcl.array]).

When typeid is applied to a glvalue expression whose type is a polymorphic class type (13.3), ...

Change in 8.3.1 [expr.unary.op] paragraph 3:

The result of the unary & operator is a pointer to its operand. The operand shall be an lvalue **of type other than "array of runtime bound" (11.3.4 [dcl.array])** or a *qualified-id*. ...

Change in 8.3.3 [expr.sizeof] paragraph 1:

... The sizeof operator shall not be applied to an expression that has function or incomplete type, **to an array of runtime bound (11.3.4 [dcl.array])**, to the parenthesized name of such types, or to a glvalue that designates a bit-field. ...

Drafting note: 8.3.7 [expr.unary.noexcept] does not need to be changed, because the declaration of an array of runtime bound cannot be lexically part of the operand of a noexcept; see also 8.1.5 [expr.prim.lambda] paragraph 2.

No change to 9.5.4 [stmt.ranged] paragraph 1, supporting runtime-sized arrays implicitly.

Insert a new paragraph before 10.1.3 [dcl.typedef] paragraph 3:

A typedef-name shall not name an array of runtime bound (11.3.4 [dcl.array]).

In a given non-class scope, a typedef specifier can be used to redefine the name of any type declared in that scope to refer to the type to which it already refers. [Example: ...]

Change in 10.1.7.2 [dcl.type.simple] paragraph 5:

For an expression *e*, the type denoted by `decltype(e)` is defined as follows:

- **if *e* has type "array of runtime bound" (11.3.4 [dcl.array]), the program is ill-formed;**
- if *e* is an unparenthesized *id-expression* naming a structured binding (11.5), ...

Change in 11 [dcl.decl] paragraph 4:

```

nptr-declarator:
    declarator-id attribute-specifier-seqopt
    nptr-declarator parameters-and-qualifiers
    nptr-declarator [ constant-expressionopt expressionopt ] attribute-specifier-seqopt
    ( ptr-declarator )

```

Drafting note: Section 11.1 [dcl.name] defining the grammar term *type-id* is intentionally unchanged. Thus, constructing an array of runtime bound in a *type-id* is ill-formed, because the grammar continues to require all *constant-expressions* in array bounds.

Change in 11.3.1 [dcl.ptr] paragraph 1:

... Similarly, the optional *attribute-specifier-seq* (7.6.1) appertains to the pointer and not to the object pointed to. **There shall be no pointers to arrays of runtime bound.**

Change in 11.3.2 [dcl.ref] paragraph 5:

There shall be no references to references, **no references to arrays of runtime bound**, no arrays of references, and no pointers to references. ...

Change in 11.3.4 [dcl.array] paragraph 1:

In a declaration *T D* where *D* has the form

```

D1 [ constant-expressionopt expressionopt ] attribute-specifier-seqopt

```

and the type of the identifier in the declaration *T D1* is "*derived-declarator-type-list T*", then the type of the identifier of *D* is an array type; if the type of the identifier of *D* contains the *auto type-specifier*, the program is ill-formed. *T* is called the array element type; this type shall not be a reference type, the (possibly cv-qualified) type void, a function type, **an array of unknown or runtime bound**, or an abstract class type.

~~If the constant-expression (8.20 [expr.const]) is present, it shall be a converted constant expression of type `std::size_t` and its value shall be greater than zero. The constant expression specifies the bound of (number of elements in) the array. If the value of the constant expression is *N*, the array has *N* elements numbered 0 to *N*-1, and the type of the identifier of *D* is "*derived-declarator-type-list array of N T*".~~ **Except as noted below, if the *expression* is omitted, the type of the identifier of *D* is "*derived-declarator-type-list array of unknown bound of T*". If the *expression* is present, it is implicitly converted to `std::size_t`. The *expression* is erroneous if:**

- **its value before converting to `std::size_t` or, in the case of an expression of class type, before application of the second standard conversion (13.3.3.1.2 `over.ics.user`) is less than or equal to zero;**
- **its value is such that the size of the allocated object would exceed the implementation-defined limit (Annex B [implimits]);**
- **the initializer of the object is a *braced-init-list* whose number of (top-level) *initializer-clauses* exceeds the number of elements to initialize; or**
- **the object is initialized with a string literal and there are more initializers than there are array elements.**

If the *expression*, after converting to `std::size_t`, is a core constant expression and the *expression* is erroneous, the program is ill-formed.

- **If the *expression*, after converting to `std::size_t`, is a core constant expression whose value is *N*, the type of the identifier of *D* is "*derived-declarator-type-list array of N T*".**

- Otherwise, the type of the identifier of D is "*derived-declarator-type-list* array of runtime bound of T" and the value of the *expression* designates the number of elements N in the array. If the *expression* is erroneous, an exception of a type that would match a handler (15.3 *except.handle*) of type `std::bad_array_length` (21.6.2.2a [*array.badlength*]) is thrown.

An object of array type contains a contiguously allocated non-empty set of N subobjects of type T. ~~Except as noted below, if the constant expression is omitted, the type of the identifier of D is "*derived-declarator-type-list* array of unknown bound of T", an incomplete object type.~~ The type "*derived-declarator-type-list* array of N T" is a different type from the type "*derived-declarator-type-list* array of unknown bound of T", see 3.9 *basic.types*. Any type of the form "*cv-qualifier-seq* array of N T" is adjusted to "array of N *cv-qualifier-seq* T", and similarly for "array of unknown bound of T" **and "array of runtime bound of T"**. The optional *attribute-specifier-seq* appertains to the array. [Example:

```
typedef int A[5], AA[2][3];
typedef const A CA;           // type is "array of 5 const int"
typedef const AA CAA;         // type is "array of 2 array of 3 const int"

void f(unsigned int n) {
    int a[n];                 // type of "a" is "array of runtime bound of int"
}
```

-- end example] [Note: ...]

Change in 11.3.4 [dcl.array] paragraph 3:

When several "array of" specifications are adjacent, a multidimensional array is created; ~~only the first of the constant expressions that specify the bounds of the arrays may be omitted.~~ In addition to ...

Add a new paragraph before 11.3.4 [dcl.array] paragraph 4:

An array of runtime bound shall only be used as the type of a local object with automatic storage duration. If the size of the array exceeds the size of the memory available for objects with automatic storage duration, the behavior is undefined [Footnote: Implementations that detect this case should throw an exception that would match a handler (18.3 [*except.handle*]) of type `std::bad_array_length` (21.6.3.2a [*array.badlength*]).] It is unspecified whether a global allocation function (6.7.4 [*basic.stc.dynamic*]) is invoked to obtain storage for the array. If it is invoked, the corresponding global deallocation function is invoked to release the storage after the lifetime of the array ended. [Footnote: Alternatively, an implementation could allocate such an array on the usual stack or on a side stack.]

Change in 11.3.5 [dcl.fct] paragraph 11:

If the type of a parameter is an array of runtime bound, the program is ill-formed. Functions shall not have a return type of type array or function, although they may have a return type of ype pointer or reference to such things. ...

Change in 11.6.1 [dcl.init.aggr] paragraph 10:

For types other than arrays of runtime bound (11.3.4 [dcl.array]), an ~~An~~ initializer-list is ill-formed if the number of *initializer-clauses* exceeds the number of members or elements to initialize. [Example: ...]

Change in 11.6.2 [dcl.init.string] paragraph 2:

[Note: There shall not be more initializers than there are array elements; **see 11.3.4 [dcl.array].** [Example:

```
char cv[4] = "asdf";           // error
```

is ill-formed since there is no space for the implied trailing '\0'. -- end example] **-- end note]**

Change in 12.2 [class.mem] paragraph 14:

~~Non-static~~ **A non-static** (12.4 [class.static]) data ~~members~~ **member** shall not have incomplete ~~types.~~ **type or type "array of runtime bound" (11.3.4 [dcl.array]).** **[Note:** In particular, a class C shall not contain a non-static member of class C, but it can contain a pointer or reference to an object of class C. **]**

Change in 17.1 [temp.param] paragraph 7:

A non-type *template-parameter* shall not be declared to have floating point, class, **array of runtime bound (11.3.4 [dcl.array])**, or void type. [Example: ...]

Drafting note: It is not necessary to explicitly prevent template argument deduction for an array of runtime bound, because 17.9.2 [temp.deduct] paragraph 8 says "If a substitution results in an invalid type or expression, type deduction fails. An invalid type or expression is one that would be ill-formed if written using the substituted arguments." 11.3.5 [dcl.fct] paragraph 11 (and other places) establishing restrictions on forming types are thus sufficient.

Change in 18.1 [except.throw] paragraph 1:

Throwing an exception transfers control to a handler. [Note: An exception can be thrown from one of the following contexts: *throw-expressions* (8.17 [expr.throw]), allocation functions (6.7.4.1 [basic.stc.dynamic.allocation]), *dynamic_cast* (8.2.7 [expr.dynamic.cast]), *typeid* (8.2.8 [expr.typeid]), *new-expressions* (8.3.4 [expr.new]), **array of runtime bound (11.8.3 [dcl.array])**, and standard library functions (20.4.1.4 [structure.specifications]). -- end note] An object is passed and the type of that object determines which handlers can catch it. [Example: ...]

Add a new section immediately before 21.6.3.2 [new.badlength]:

21.6.3.2a Class bad_array_length [array.badlength]

```
namespace std {  
    class bad_array_length : public bad_alloc {  
    public:  
        bad_array_length() noexcept;  
        const char* what() const noexcept override;  
    };  
}
```

The class bad_array_length defines the type of objects thrown as exceptions by the implementation to report an attempt to allocate an array of runtime bound with a size less than or equal to zero or greater than an implementation-defined limit (11.3.4 [dcl.array]).

bad_array_length() noexcept;

Effects: constructs an object of class bad_array_length.

const char* what() const noexcept override;

Returns: An implementation-defined NTBS.

Remarks: The message may be a null-terminated multibyte string (20.4.2.1.5.2

[[multibyte.strings](#)]), suitable for conversion and display as a `wstring` (24.3 [[string.classes](#)], 25.4.1.4 [[locale.codecvt](#)]).

Add a new section immediately after 21.9 [[support.initlist](#)]:

21.10 Stack-based arrays [[support.stack_array](#)]

The header `<stack_array>` defines a class template for an array type which can only be used for variables of automatic storage duration and whose size is defined at construction. A `stack_array` is a contiguous container (26.2.1 [[container.requirements.general](#)]). All functions specified in this subclause are signal-safe (21.10.4 [[support.signal](#)]).

A `stack_array` satisfies all of the requirements of a container (26.2 [[container.requirements](#)]), except that default construction and swap are not supported. A `stack_array` satisfies some of the requirements of a sequence container (26.2.3 [[sequence.reqmts](#)]). Descriptions are provided here only for operations on `stack_array` that are not described in one of these tables and for operations where there is additional semantic information.

21.10.1 Header `<stack_array>` synopsis [[stack_array.syn](#)]

```
template<class T>
class stack_array {
    using value_type      = T;
    using pointer         = T*;
    using const_pointer   = const T*;
    using reference       = T&;
    using const_reference = const T&;
    using size_type       = size_t;
    using difference_type = ptrdiff_t;
    using iterator        = implementation-defined ;
    using const_iterator  = implementation-defined ;

    explicit stack_array(size_type n);
    stack_array(size_type n, const T& value);
    stack_array(const stack_array&);
    stack_array& operator=(const stack_array&);

    // iterators:
    constexpr iterator      begin() noexcept;
    constexpr const_iterator begin() const noexcept;
    constexpr iterator      end() noexcept;
    constexpr const_iterator end() const noexcept;
    constexpr const_iterator cbegin() const noexcept;
    constexpr const_iterator cend() const noexcept;

    // capacity:
    constexpr bool          empty() const noexcept;
    constexpr size_type size() const noexcept;
    constexpr size_type max_size() const noexcept;

    // element access:
    constexpr reference      operator[](size_type n);
    constexpr const_reference operator[](size_type n) const;
    constexpr reference      at(size_type n);
    constexpr const_reference at(size_type n) const;
    constexpr reference      front();
    constexpr const_reference front() const;
    constexpr reference      back();
    constexpr const_reference back() const;

    constexpr T *           data() noexcept;
```

```
constexpr const T * data() const noexcept;
};
```

21.10.2 Constructors [stack_array.cons]

Declaring or creating an object of type `stack_array` is ill-formed unless that object is a variable with automatic storage duration. The destructor of a `stack_array` shall not be invoked explicitly (8.2.2 [expr.call]), but only implicitly (9.6 [stmt.jump]).

```
stack_array(size_type n);
```

Effects: Constructs a `stack_array` with `n` default-constructed elements.

Requires: `n` is greater than 0. `T` shall be `DefaultConstructible`.

Complexity: Linear in `n`.

```
stack_array(size_type n, const T& value);
```

Effects: Constructs a `stack_array` with `n` copies of `value`.

Requires: `n` is greater than 0. `T` shall be `CopyConstructible`.

Complexity: Linear in `n`.

21.10.3 Member functions [stack_array.mem]

```
constexpr size_type size() const noexcept;
```

Returns: The number of elements `n` used for construction.

```
constexpr T* data() noexcept;
constexpr const T* data() const noexcept;
```

Returns: A pointer such that `data() == addressof(front())`, and `[data(), data() + size())` is a valid range.